Evidentia

A Research Claim Auditor with a Claim Ladder

Detailed idea and step-by-step build guide

OncoSys Cancer AI Hackathon · Track 2: Evidence Navigator · Featured product: Research Claim Auditor

Prepared November 12, 2026 · Code freeze Saturday, November 14, 2:15 PM ET

One-line pitch: You give Evidentia a research report. For every sentence it tells you whether the data supports it, how solid that answer is, and the strongest honest way to say it, then records the decision and who must sign off.

Name note: "Evidentia" is a working name. Other options: *claim-ceiling*, *warrant-audit*, *research-claim-assurance*. Replace the name everywhere once the team decides.

Contents

1. The idea in plain words
2. How judging works and how we target it
3. What the user sees, screen by screen
4. How the engine works: checks, verdicts, robustness, Claim Ladder, decision rules
5. Role views and safety rules
6. Outputs: audit.json and the Decision Record
7. Architecture and folder structure
8. Step-by-step build instructions (phases 0-6)
9. Team roles and timeline
10. Testing plan
11. Demo script and video
12. Submission checklist
13. Risks and fallbacks
14. Appendix: data facts and commands

1. The idea in plain words

The problem. Research reports about cancer data often say things the data can't fully back up: a percentage that's slightly wrong, a claim about treatment when the data has no treatment records, or a result that only holds if you count one particular way. A reviewer asked "can we trust this report?" would have to dig through files and code to find out. Most people can't.

The user. A **data-use reviewer** who has to decide whether a report can be published. Other readers of the same result are a data engineer, a researcher and a program leader.

What Evidentia does, in four steps:

1. **Reads the report** and pulls out every sentence that makes a checkable factual claim.
2. **Checks each claim against the real TCGA-BRCA data** with ordinary code (not AI guesses), and gives one of three verdicts: **supported**, **contradicted**, or **insufficient evidence**.
3. **Tests how solid each verdict is** by redoing the check several reasonable ways (for example a different subtype label, or women only). If the verdict never changes it's **robust**; if it flips it's **fragile**, and the tool names the choice that flips it.
4. **Shows the Claim Ladder:** the original sentence with over-strong words crossed out, and the strongest honest version underneath. Then it suggests one overall decision (Proceed, Fix, Narrow or Stop) that a person confirms, and exports the record.

Example, using the organizers' practice report:

Claim in the report	Verdict	What Evidentia shows
535 of 1,101 primary tumors have all five data layers	Supported, robust	Count recomputed from sample_manifest.csv: 535 of 1,101
Luminal A is about 62% of subtyped primary tumors	Contradicted, robust	~~about 62%~~ **49.9% (421 of 844)**. Never 62% under any reasonable choice (44.3% with the 2012 subtype label)
Patients with highest ER-alpha protein responded best to hormone therapy	Insufficient evidence	The data has no treatment or response records. Would need: treatment and outcome data. Also flags "responded best" and "guide treatment" as clinical language

Why it's different: other claim checkers say true or false. Evidentia also says **how solid** the answer is and **what you are allowed to say instead**. That turns a list of problems into a decision.

2. How judging works and how we target it

What we learned from the challenge repo:

- **Hidden input:** on Saturday the judges hand you an envelope number and a four-word code. Opening it gives `claim_auditor_report.md`, a new report with different errors. You run the tool on it live.
- **Answer key:** you must produce `audit.json` in the shared format. The organizers' checker compares your verdicts with their answer key, and judges see the comparison while scoring.
- **Live demo requirement:** run the practice report; the judge opens the calculation behind the supported claim, the correct number behind the contradicted one, and the explanation of what's missing for the claim the data can't answer.
- **Surprise:** judges may add a new user role or a conflicting finding. The tool must cope without the presenter explaining.

- **Key hint:** the schema's own example for `correct_value` is "16.7% (141 of 844)", the Basal share from reference analysis A2. So **the answer key likely uses the definitions in reference_analyses.py**. Those become our primary numbers; the other ways of counting are for the robustness check only.

Rubric map

Criterion	Pts	Where it's earned in Evidentia
Decision usefulness	20	Decision screen (Proceed / Fix / Narrow / Stop) with reasons, plus a corrected, publishable sentence for each claim
Understanding the evidence	20	Every number opens an evidence panel: sample IDs, file, checksum, exact code. Breadcrumb keeps the user's place
Role fit	15	Four role views (plus add-a-role) reading the same fact table, so numbers never differ between views
Safety and uncertainty	15	Robust/fragile tags, insufficient-evidence explanations, crossed-out over-strong words, human-review gate, research-only banner
Usability and accessibility	15	Plain words, verdict shown as text plus icon (not colour alone), keyboard navigation, glossary tooltips
Working tool and setup	10	Python + Streamlit, 3 setup commands, runs with no AI key
Communication	5	3-minute demo built around one story: "the report said 62%; it's really 50%"

3. What the user sees, screen by screen

Screen	What's on it	What the user does
1. Start	Upload box for a report (.md or .txt), the data folder in use with its checksum status (all files verified), and a role picker	Uploads the report, picks a role
2. Claims list	Each sentence from the report, marked as a checkable claim or not. Checkable ones show the claim type that was detected	Ticks or unticks claims if extraction got one wrong
3. Audit board (main screen)	One card per claim: verdict badge, robust/fragile tag, and the Claim Ladder (crossed-out words and the honest version)	Scans results; clicks any card
4. Evidence panel	Calculation in words, the numbers, sample count, list of sample IDs, source file and checksum, the exact code with a copy button, and the robustness table	Verifies the result; reruns with a changed rule
5. Review queue	For each claim: accept or override the verdict, with a reason and reviewer role. Both the tool's verdict and the person's are kept	Confirms or overrides each verdict
6. Decision	Suggested overall decision with the rule that produced it, the limitations, who must sign off, and "what would change this decision"	Chooses an option, writes a reason, exports

A banner on every screen: **"Research prototype. Not a clinical decision system. Verdicts need human review."**

4. How the engine works

4.1 Claim types and their checks

Each extracted claim is matched to a **check template**: a small, tested Python function. The practice report and the product brief suggest the hidden report will use these kinds of claims:

Claim type	Example	How it's checked
Count	"535 of 1,101 primary tumors have all five layers"	Filter sample_manifest.csv; count rows. Compare to the stated number
Share / percentage	"Luminal A is about 62% of subtyped tumors"	Same definition as reference analysis A2 (primary tumors with a PAM50Call_RNAseq value); numerator / denominator
Group comparison	"ER-alpha protein is higher in ER-positive tumors"	Same definition as A1; compare group means; report both group sizes and the difference
Agreement / conflict	"Clinical HER2 status matches HER2 protein"	Cross-tabulate clinical label vs protein level; count disagreements
Coverage / missing data	"Every primary tumor has protein data"	Count has_protein in the manifest; list the missing samples
Not answerable	"...responded best to hormone therapy"	Detected by keywords (treatment, response, cause, predict, survival benefit) plus a list of what the data does not contain. Verdict: insufficient evidence, with what data would be needed

4.2 Verdict rules (written down, so they're repeatable)

Situation	Verdict
Stated number equals our number after rounding to the report's precision. For "about" / "roughly", within 2 percentage points or 2% of a count	Supported
Stated number outside that tolerance	Contradicted, with correct_value such as "49.9% (421 of 844)"
Direction claims ("higher", "more"): direction matches and the group difference is clearly non-zero	Supported; otherwise contradicted
The data has no field that can test the claim, or the claim is causal, predictive or about treatment	Insufficient evidence, plus "would need: ..."
The claim mixes a checkable part and an unanswerable part	Split into two claims, or insufficient evidence for the whole sentence, with the reason

Important: the primary number always uses the reference_analyses.py definitions, which are likely what the answer key uses. Alternative definitions never change the verdict; they only set the robust or fragile tag.

4.3 Robustness check ("count it differently")

For share and comparison claims, rerun the check under small, reasonable changes. Keep it to three choices (8 versions):

Choice	Option A (primary)	Option B
Subtype label	PAM50Call_RNAseq (as in A2)	PAM50_mRNA_nature2012, with names mapped, e.g. "Luminal A" to LumA
Sex	Everyone	Female only (13 male, 5 unknown excluded)
Sample set	Primary tumors (code 01)	Primary tumors that have all five data layers

- **Robust:** the verdict is the same in all 8 versions.

- **Fragile:** the verdict changes in some versions. The tool names the choice, e.g. "flips when using the 2012 subtype label".
- **Skeptic tests (comparison claims only):** (1) shuffle the group labels 200 times; the real difference should be far larger than the shuffled ones. (2) Leave out one hospital at a time (tissue-source-site code in the barcode, 40 sites); if the result depends on one site, flag it.

Real example: the Luminal A share is 49.9% (RNA-seq label), 49.9% (women only) and 44.3% (2012 label). The claim of 62% is contradicted in every version, so it's **contradicted, robust**.

4.4 Claim Ladder (the strongest honest wording)

Built from fixed rules, not AI, so it gives the same output every time:

Rule	Words or patterns	What the ladder does
Wrong number	Any number that is contradicted	Cross it out; insert the correct value with its denominator
Vague amount	about, most, nearly all, majority	Replace with the exact count, e.g. "421 of 844"
Clinical or treatment	responded, therapy, treatment, guide treatment, patients benefit	Cross out; mark the claim as not answerable from this data
Cause or prediction	causes, drives, predicts, determines	Replace with "was associated with" or mark as not answerable
Over-generalising	breast cancer patients, all women, in general	Add the scope: "in this TCGA-BRCA release, among N primary tumors"

Example: ~~about 62% of~~ **49.9% (421 of 844) of** primary tumors with an RNA-seq subtype call in this TCGA-BRCA release were Luminal A.

4.5 Overall decision rules

The tool **suggests** a decision; the reviewer chooses and must give a reason.

If...	Suggested decision
Data files fail their checksum, or the report is about a different release	Stop
Any claim is contradicted	Fix a problem (correct the numbers before publishing)
No contradictions, but some claims are insufficient evidence or fragile	Narrow the question (remove or qualify those claims)
All claims are supported and robust	Proceed (still needs sign-off)

Review gate: the export button stays disabled until every claim has a human accept or override and a reviewer role is named. **Conflicts** (for example the tool says supported but a second source or finding disagrees) are shown side by side and must be resolved by a person.

5. Role views and safety rules

All roles read one shared fact table, so a number can never differ between views. Only the wording and the level of detail change.

Role	Sees first	Detail available
Program leader	One sentence: the decision, the main risk, the next step	Counts by verdict; who signs

Role	Sees first	Detail available
Data-use reviewer	Which claims can be published as written, which must change, who signs off	Review queue, review gate, Decision Record
Researcher	Corrected wording, robust/fragile tags, alternative numbers	Robustness table, skeptic tests
Data engineer	Sample IDs, files, checksums, exact code	Copy code; rerun from the command line
New role (added live)	Chosen from a form: which sections, how much detail, which words to explain	Saved in roles.yaml without changing code

Safety rules (always on)

- Always show counts with their denominator ("421 of 844"), never a bare percentage.
- Missing values are shown as "unknown", never dropped silently or treated as zero.
- Every statement is labelled **Computed** (from the data), **Reported** (taken from the report) or **Interpretation**.
- No diagnosis, treatment advice or risk scores. Clinical wording in a report is flagged, never repeated as fact.
- If an AI model is used, it only helps split the report into claims. Any number it writes must match the fact table or it is blocked. Verdicts come only from code.
- No API keys in the repo. The tool works fully without AI.

6. Outputs

6.1 audit.json (required, shared format v0.2)

One finding per claim with `kind: "claim_check"`. Required fields: id, summary, severity, confidence, evidence. Claim fields: claim_text, claim_verdict, correct_value (contradicted only). Top level needs format_version, producer, data (release and file checksums), question and findings.

```
{
  "format_version": "0.2",
  "producer": {"team": "<team>", "product": "Research Claim Auditor",
               "track": "evidence-navigator"},
  "data": {"dataset": "TCGA-BRCA", "releases": [{"name": "A",
           "files": [{"file": "clinical.tsv", "sha256": "<64 hex chars>"}]}]},
  "question": "Are the claims in practice_brief.md supported by release A?",
  "verdict": "Fix a problem",
  "findings": [{
    "id": "C2", "kind": "claim_check",
    "claim_text": "Luminal A is the largest RNA subtype, accounting for about 62% ...",
    "claim_verdict": "contradicted",
    "correct_value": "49.9% (421 of 844)",
    "summary": "Luminal A share is 49.9%, not about 62%.",
    "severity": "medium", "confidence": "high",
    "evidence": [{"kind": "file", "id": "clinical.tsv", "release": "A",
                  "note": "primary tumors with PAM50Call_RNAseq; robust in 8 of 8 versions"}],
    "limitation": "Subtype calls exist for 844 of 1,101 primary tumors.",
    "needs_human_review": true
  }]
}
```

Always run `python format/validate.py audit.json` before showing or submitting it.

6.2 Decision Record (Markdown, for people)

- Report name, data release, checksums, date, and the tool version.

- The chosen decision, the reviewer's reason, and the rule the tool used to suggest it.
- Table of claims: original text, verdict, robust/fragile, corrected wording, and any human override with its reason.
- Important limitations (e.g. only 844 of 1,101 tumors have a subtype call).
- Who must sign off (e.g. data-use reviewer and principal investigator).
- What would change this decision (e.g. "if treatment-response data were added, claim 3 could be tested").

7. Architecture and folder structure

A straight line from inputs to outputs, with one shared fact table in the middle:

Step	Module	Does what
1. Load	loader.py	Reads a release folder (data/tcga-brca/raw or an envelope's release/), verifies checksums, loads clinical, protein, survival and the manifest
2. Extract	extract.py	Splits the report into sentences; finds numbers and keywords; assigns a claim type. Optional AI helper behind a flag
3. Check	checks/*.py	One function per claim type. Returns a Fact: value, numerator, denominator, sample IDs, code used
4. Judge	verdict.py	Applies the verdict rules and tolerance
5. Stress	robustness.py	Reruns checks under the 8 versions; skeptic tests
6. Word	ladder.py	Builds the crossed-out and honest wording
7. Decide	decision.py	Suggests Proceed / Fix / Narrow / Stop
8. Export	export.py	Writes audit.json (validated) and decision_record.md
UI	app.py	Streamlit screens; reads only the fact table, never recalculates

```
evidentia/
  README.md            short summary, setup, how to run, demo video link
  requirements.txt     pinned versions
  app.py               Streamlit interface
  cli.py               python cli.py audit <report.md> --release <folder> --out audit.json
  roles.yaml           role views (editable, add a role here or in the app)
  evidentia/
    loader.py  extract.py  verdict.py  robustness.py
    ladder.py  decision.py  export.py  facts.py
    checks/  count.py  share.py  compare.py  agreement.py  coverage.py  unanswerable.py
  tests/
    test_practice_brief.py   expected verdicts for the practice report
    reports/                 our own test reports with known answers
  outputs/                   (git-ignored) audit.json, decision_record.md
  LICENSE  THIRD_PARTY.md
```

8. Step-by-step build instructions

Each phase ends with a **done when** test. Don't start the next phase until it passes.

Phase 0: Setup (about 30 minutes)

1. Create the GitHub repo and add teammates.
2. Clone the challenge repo next to it and fetch the data: `python data/tcga-brca/fetch.py`.

3. Create a virtual environment and install pandas, streamlit, jsonschema, pyyaml. Pin versions in requirements.txt.
4. Make the tool take the release folder as an argument (it must work with `envelopes/opened/<E>/release/`).

Done when: `python -c "import evidentia.loader"` loads all files and prints checksums OK.

Phase 1: Checks engine (Thursday; most important)

1. Write loader.py and a Fact data class (value, numerator, denominator, sample_ids, code_text, file, sha256).
2. Write the count, share and compare checks using the exact definitions in reference_analyses.py.
3. Write unanswerable.py: keyword list plus a list of what the data does not contain (treatment, response, recurrence cause, etc.).
4. Write verdict.py with the tolerance rules from section 4.2.
5. Hard-code the 3 practice claims and confirm the expected verdicts.

Done when: the practice report gives: 535 supported; 62% contradicted with 49.9% (421 of 844); hormone therapy insufficient evidence. Post this as Checkpoint 1.

Phase 2: Extraction and audit.json (Thursday night)

1. extract.py: split into sentences; keep sentences with a number, a comparison word or a clinical keyword.
2. Map each to a claim type with simple patterns (percent sign means share; "of N" means count; "higher/lower" means comparison).
3. export.py: write audit.json and run format/validate.py automatically.
4. cli.py: one command from report to audit.json.

Done when: `python cli.py audit practice_brief.md --release data/tcga-brca/raw` writes a valid audit.json in under 60 seconds.

Phase 3: Interface (Friday morning)

1. Streamlit app with screens 1-6 from section 3.
2. Audit board cards: verdict as text plus icon plus colour (never colour alone).
3. Evidence panel with the code, sample IDs (downloadable CSV), file and checksum.
4. Review queue with accept/override and reason; review gate on export.

Done when: a teammate who didn't build it can audit the practice report and export without help.

Phase 4: Robustness and Claim Ladder (Friday afternoon)

1. robustness.py: the 8 versions, with the subtype name mapping ("Luminal A" to LumA, etc.).
2. Skeptic tests for comparison claims: label shuffle and leave-one-site-out.
3. ladder.py: the five rules from section 4.4.
4. Show the robustness table in the evidence panel and the tag on each card.

Done when: the 62% claim shows "contradicted, robust (8 of 8)" and the ladder rewrite.

Phase 5: Roles, decision and record (Friday evening)

1. roles.yaml with four roles; an "Add a role" form that writes to it.
2. decision.py with the rules from section 4.5; decision screen with "what would change this decision".
3. Decision Record export (Markdown).
4. Repo swap with the paired team: they follow the README alone. Fix everything they trip on.

Done when: the paired team runs it from a fresh clone and completes the scenario.

Phase 6: Polish and rehearse (Saturday morning, before 2:15 PM)

1. Accessibility pass: keyboard-only run-through; Lighthouse check; plain-language glossary.
2. Write 3 of our own test reports with different errors; fix any wrong verdicts.
3. Rehearse the envelope run: `python envelopes/unseal.py practice orange meadow pilot lantern`, then run the tool. Time it.
4. Finish the README, record the demo video, run a secret scan, freeze the code.

Done when: full run from unsealing to exported audit.json in under 2 minutes, three times in a row.

9. Team roles and timeline

Role	Owns	Phases
Engine lead	loader, checks, verdict rules, tests	1, 2, 4
Interface lead	Streamlit screens, evidence panel, accessibility	3, 5, 6
Robustness and wording	robustness.py, skeptic tests, Claim Ladder rules	4
Output and docs	audit.json, validator, Decision Record, README, demo video	2, 5, 6
Tester (anyone, rotating)	Writes test reports with known answers, tries to break the tool	all

With 3 people, merge "robustness and wording" into engine, and "output and docs" into interface.

When	Milestone
Thu Nov 12, by evening	Phase 0-1 done; Checkpoint 1 screenshot posted
Thu Nov 12, night	Phase 2: CLI produces a valid audit.json
Fri Nov 13	Phases 3-5; Checkpoint 2 repo swap; fix setup issues
Sat Nov 14, 10:00-2:15	Phase 6; code freeze 2:15 PM; demos 2:30-4:00 PM

10. Testing plan

- **Practice report test** (automated): the three expected verdicts in section 1 must pass on every commit.
- **Reference match test:** our share and comparison checks must reproduce reference_analyses.py exactly (A1: 529 ER-positive, 169 ER-negative, difference 3.051; A2: 844 subtyped, LumA 421).

- **Our own reports:** write 3 short reports with 4-5 claims each, covering every claim type, including approximate numbers ("about", "nearly"), ranges, and one mixed claim.
- **Trap tests:** subtype name mismatch, male samples, normal tissue (code 11) mixed in, a claim about RNA when the envelope release has no RNA file (the tool must say so, not crash).
- **Stranger test:** someone outside the team uses it without help. Note where they hesitate.
- **Timing test:** unseal, run and export in under 2 minutes.

11. Demo script (3 minutes) and video

Time	What happens on screen	What to say
0:00-0:20	Start screen	"Reviewers get reports like this and must decide: can we publish it? Evidentia checks every claim against the real data."
0:20-0:50	Upload practice report; audit board appears	"Three claims: one supported, one wrong, one the data can't answer."
0:50-1:40	Open the 62% claim; evidence panel; robustness table	"The report said 62%. It's 49.9%, 421 of 844, and it's never 62% however you count it. Here's the code and the sample IDs."
1:40-2:10	Open the hormone-therapy claim	"The data has no treatment records, so we don't guess. We say what data would be needed."
2:10-2:40	Switch role to program leader; then the decision screen	"Same facts, simpler view. The tool suggests Fix a problem; the reviewer confirms with a reason."
2:40-3:00	Export audit.json and Decision Record	"Validated, with who must sign off. Research prototype, never a clinical decision."

Video: record the same script (2-3 minutes) with a screen recorder. Start with the problem, the user and the decision; end with the outcome. Put the link at the top of the README.

12. Submission checklist

- Public GitHub repo with no secrets, keys or private links (run a secret scanner such as Gitleaks).
- No raw TCGA files committed (they're fetched); outputs/ and envelopes/opened/ are git-ignored.
- README: one-paragraph summary, the user and decision, setup in 3 commands, how to run on an envelope, demo video link, limitations, data credits (TCGA Research Network, UCSC Xena, NCI IDC).
- requirements.txt with pinned versions; tested from a fresh clone on another machine.
- audit.json validates with format/validate.py.
- Research-only banner on every screen; no clinical claims anywhere, including the README.
- THIRD_PARTY.md listing libraries and their licenses; LICENSE file for our code.
- Short demo video uploaded and linked.

13. Risks and fallbacks

Risk	Fallback
Hidden report uses a claim type we didn't build	Default to insufficient evidence with a clear explanation; the reviewer can override. Never guess
Claim extraction misses or splits a sentence badly	Reviewer can tick, untick or edit claims on screen 2
Our numbers differ from the answer key's definitions	Primary numbers follow reference_analyses.py; show alternative definitions as robustness only
Envelope release has no RNA file	Use the manifest for layer coverage; state clearly when a claim needs RNA data that isn't present
Interface not finished	The CLI alone still produces a valid audit.json for the answer key
Setup fails on a judge's machine	Pinned versions, tested on the paired team's machine, and a short troubleshooting section in the README

Appendix: data facts and commands

Fields we confirmed in release A

What	Where	Notes
Samples / patients	sample_manifest.csv	1,247 samples, 1,101 patients; 1,101 primary tumors
Data layers present	sample_manifest.csv	has_survival, has_mutation, has_copy_number, has_rna, has_protein; 535 primary tumors have all five
Subtype (primary)	clinical.tsv: PAM50Call_RNAseq	844 primary tumors called; labels LumA, LumB, Basal, Her2, Normal
Subtype (alternative)	clinical.tsv: PAM50_mRNA_nature2012	522 called; labels spelled "Luminal A" etc. (needs mapping)
ER / PR status	breast_carcinoma_estrogen_receptor_status, ..._progesterone_receptor_status	Positive, Negative, other values
HER2 status	HER2_Final_Status_nature2012, lab_proc_her2_neu_immunohistochemistry_receptor_status	
Sex	clinical.tsv: gender	1,229 female, 13 male, 5 unknown
Hospital (site)	barcode characters 6-7, or tissue_source_site	40 sites
Protein	protein_rppa.tsv.gz, row ERALPHA etc.	Rows are proteins, columns are samples
Survival	survival.tsv: OS, PFI, DSS, DFI (+ .time)	Not needed for the core product
Not in the data	-	Treatment given, treatment response, causes. Claims about these are insufficient evidence

Useful commands

```
python data/tcga-brca/fetch.py                       # get release A (verified)
python data/tcga-brca/reference_analyses.py          # reference numbers A1, A2
python envelopes/unseal.py practice orange meadow pilot lantern   # rehearsal
python cli.py audit <report.md> --release <folder> --out outputs/audit.json
python format/validate.py outputs/audit.json         # must pass
streamlit run app.py                                 # the interface
```

Remember the bar: a judge can use the workflow without us narrating, sees one result they can verify, and one limitation they can inspect. A simple tool that runs perfectly beats an ambitious one that crashes.